DEPTH-FIRST SEARCH

DEPTH-FIRST SEARCH

Input: $G = (V, E)$, directed or undirected. No source vertex given.

Output:

- 2 ***timestamps*** on each vertex:
  - $v.d =$ ***discovery time***
  - $v.f =$ ***finish time***

  These will be useful for other algorithms later on.
- $v.\pi$ is v's predecessor in the ***depth-first forest*** of ≥ 1 ***depth-first trees***. If $u = v.\pi$, then (u, v) is a ***tree edge***.

DEPTH-FIRST SEARCH (continued)

Methodically explores *every* edge.

- Start over from different vertices as necessary.

As soon as a vertex is discovered, explore from it.

- Unlike BFS, which puts a vertex on a queue so that it's explored from later.

DEPTH-FIRST SEARCH (continued)

As DFS progresses, every vertex has a ***color***:

- WHITE = undiscovered
- GRAY = discovered, but not finished (not done exploring from it)
- BLACK = finished (have found everything reachable from it)

DEPTH-FIRST SEARCH (continued)

As DFS progresses, every vertex has a ***color***:

- WHITE = undiscovered
- GRAY = discovered, but not finished (not done exploring from it)
- BLACK = finished (have found everything reachable from it)

Discovery and finish times:

- Unique integers from 1 to $2|V|$.
- For all v, $v.d < v.f$.

In other words, $1 \leq v.d < v.f \leq 2|V|$.

PSEUDOCODE

```
DFS(G)
1   for each vertex u ∈ G.V
2       u.color = WHITE
3       u.π = NIL
4   time = 0
```

Global search starts a local search on each vertex to explore entire graph.

PSEUDOCODE

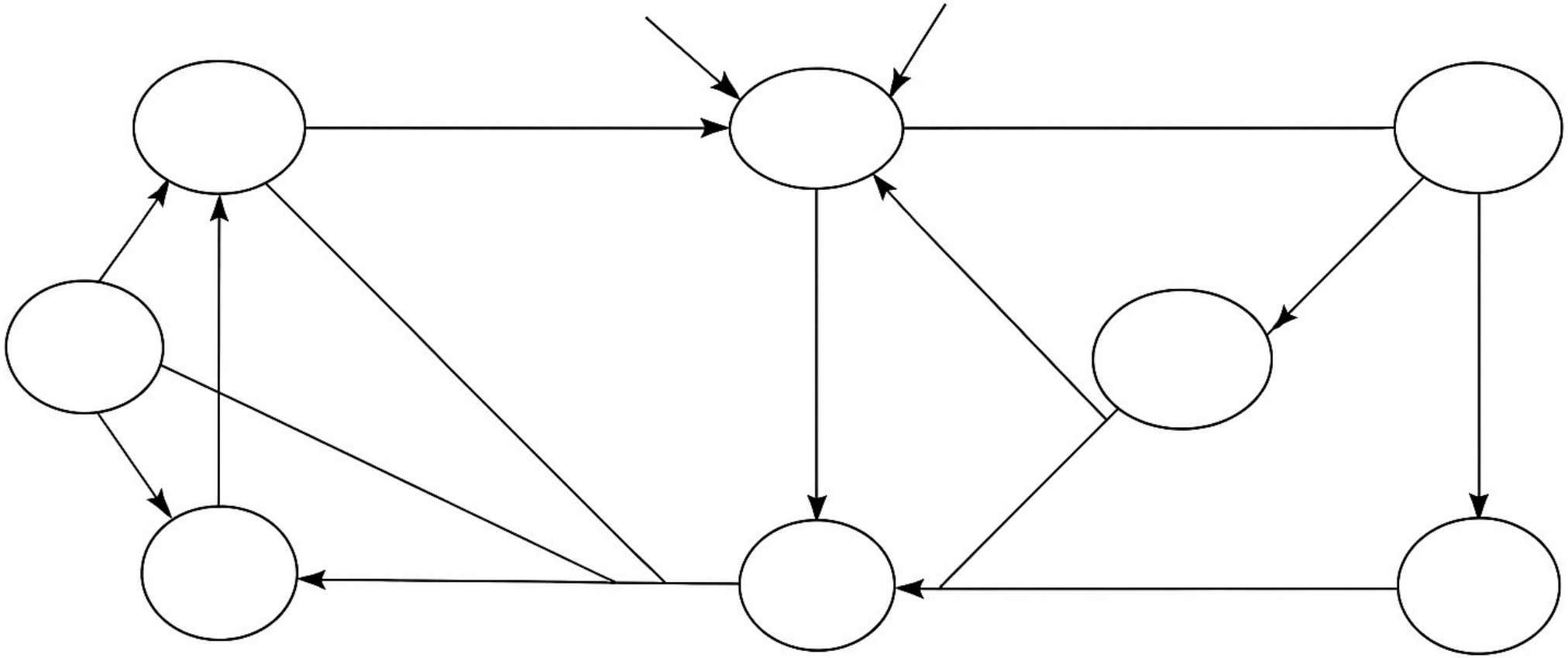
```
DFS(G)
1   for each vertex u ∈ G.V
2       u.color = WHITE
3       u.π = NIL
4   time = 0
5   for each vertex u ∈ G.V
6       if u.color == WHITE
7           DFS-VISIT(G, u)
```

Global search starts a local search on each vertex to explore entire graph.

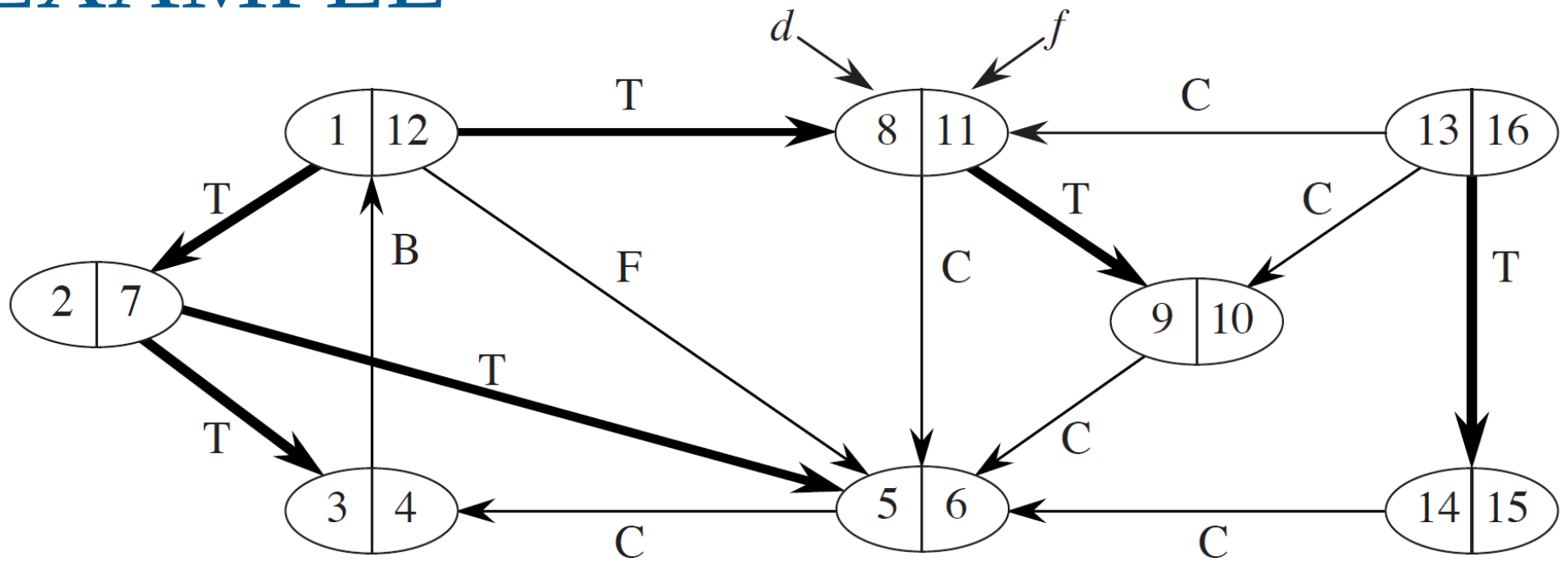
PSEUDOCODE (continued)

```
DFS-VISIT(G, u)
 1   time = time + 1                    // white vertex u has just been discovered
 2   u.d = time
 3   u.color = GRAY
 4   for each vertex v in G.Adj[u]      // explore each edge (u, v)
 5       if v.color == WHITE
 6           v.π = u
 7           DFS-VISIT(G, v)
 8   time = time + 1
 9   u.f = time
10   u.color = BLACK                    // blacken u; it is finished
```

EXAMPLE

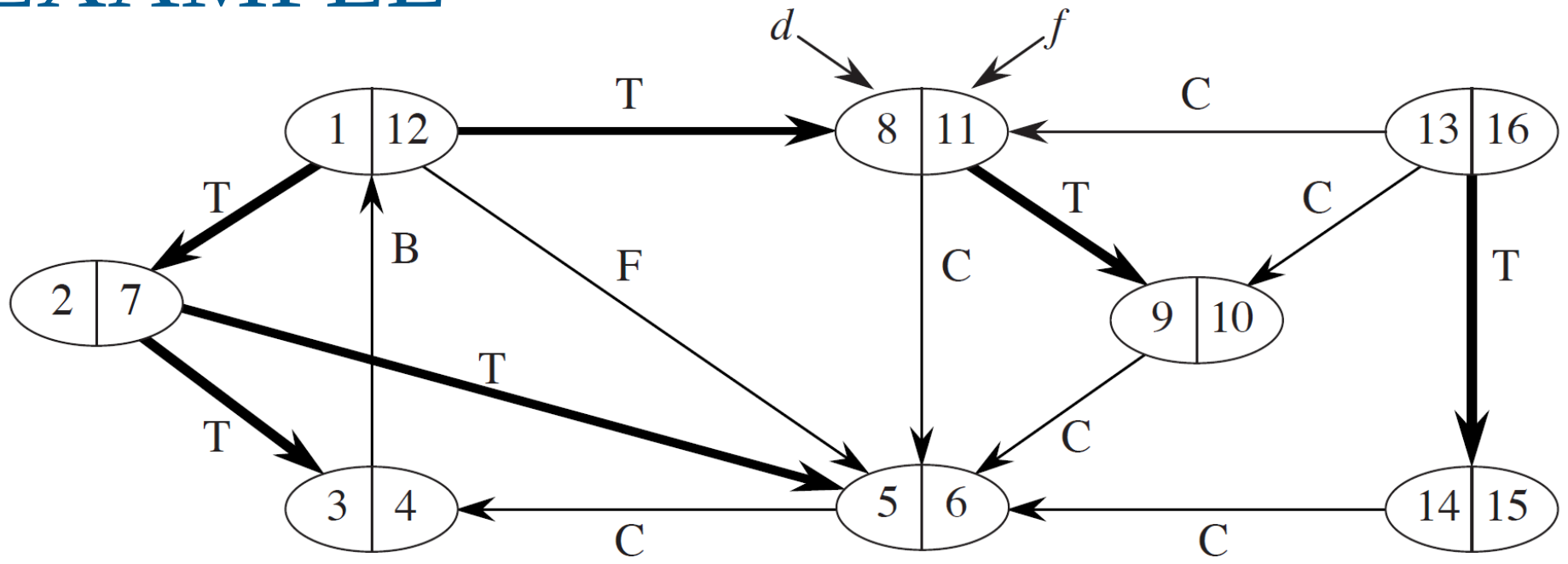


EXAMPLE



Time $= \Theta(V + E)$.

EXAMPLE



Time $= \Theta(V + E)$.

- Similar to BFS analysis.
- Θ, not just O, since guaranteed to examine every vertex and edge.

Each depth-first tree is made of edges (u, v) such that u is gray and v is white when (u, v) is explored.

THEOREM (PARENTHESIS THEOREM)

For all u, v, exactly one of the following holds:

1. $u.d < u.f < v.d < v.f$ or $v.d < v.f < u.d < u.f$

THEOREM (PARENTHESIS THEOREM)

For all u, v, exactly one of the following holds:

1. $u.d < u.f < v.d < v.f$ or $v.d < v.f < u.d < u.f$ (i.e., the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint) and neither of u and v is a descendant of the other.

THEOREM (PARENTHESIS THEOREM)

For all u, v, exactly one of the following holds:

1. $u.d < u.f < v.d < v.f$ or $v.d < v.f < u.d < u.f$ (i.e., the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint) and neither of u and v is a descendant of the other.
2. $u.d < v.d < v.f < u.f$ and v is a descendant of u. (v is discovered after and finished before u.)

THEOREM (PARENTHESIS THEOREM)

For all u, v, exactly one of the following holds:

1. $u.d < u.f < v.d < v.f$ or $v.d < v.f < u.d < u.f$ (i.e., the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint) and neither of u and v is a descendant of the other.
2. $u.d < v.d < v.f < u.f$ and v is a descendant of u. (v is discovered after and finished before u.)
3. $v.d < u.d < u.f < v.f$ and u is a descendant of v. (u is discovered after and finished before v.)

THEOREM (PARENTHESIS THEOREM)

For all u, v, exactly one of the following holds:

1. $u.d < u.f < v.d < v.f$ or $v.d < v.f < u.d < u.f$ (i.e., the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint) and neither of u and v is a descendant of the other.
2. $u.d < v.d < v.f < u.f$ and v is a descendant of u. (v is discovered after and finished before u.)
3. $v.d < u.d < u.f < v.f$ and u is a descendant of v. (u is discovered after and finished before v.)

So $u.d < v.d < u.f < v.f$ (v is both discovered and finished after u) *cannot* happen.

THEOREM (PARENTHESIS THEOREM)

For all u, v, exactly one of the following holds:

1. $u.d < u.f < v.d < v.f$ or $v.d < v.f < u.d < u.f$ (i.e., the intervals $[u.d, u.f]$ and $[v.d, v.f]$ are disjoint) and neither of u and v is a descendant of the other.
2. $u.d < v.d < v.f < u.f$ and v is a descendant of u. (v is discovered after and finished before u.)
3. $v.d < u.d < u.f < v.f$ and u is a descendant of v. (u is discovered after and finished before v.)

So $u.d < v.d < u.f < v.f$ (v is both discovered and finished after u) *cannot* happen.

OK:	( ) [ ]	([ ])	[( )]
Not OK:	([ )]	[( ])	

CLASSIFICATION OF EDGES

- ***Tree edge:*** in the depth-first forest. Found by exploring (u, v).
- ***Back edge:*** (u, v), where u is a descendant of v.
- ***Forward edge:*** (u, v), where v is a descendant of u, but not a tree edge.
- ***Cross edge:*** any other edge. Can go between vertices in same depth-first tree or in different depth-first trees.

In an undirected graph, there may be some ambiguity since (u, v) and (v, u) are the same edge. Classify by the first type above that matches.

Lemma

In a depth-first search of an undirected graph G, every edge of G is either a tree edge or a back edge.

Proof:

- Consider an arbitrary edge (u,v). WLOG let u.d < v.d

Lemma

In a depth-first search of an undirected graph G, every edge of G is either a tree edge or a back edge.

Proof:

- Consider an arbitrary edge (u,v). WLOG let u.d < v.d

- Property between u and v? (in terms of times)

- First time the search explores the edge (u,v):

    From u to v: tree edge

    From v to u: back edge

DAG

Directed acyclic graph (dag)

A directed graph with no cycles.

DAG

Directed acyclic graph (dag)

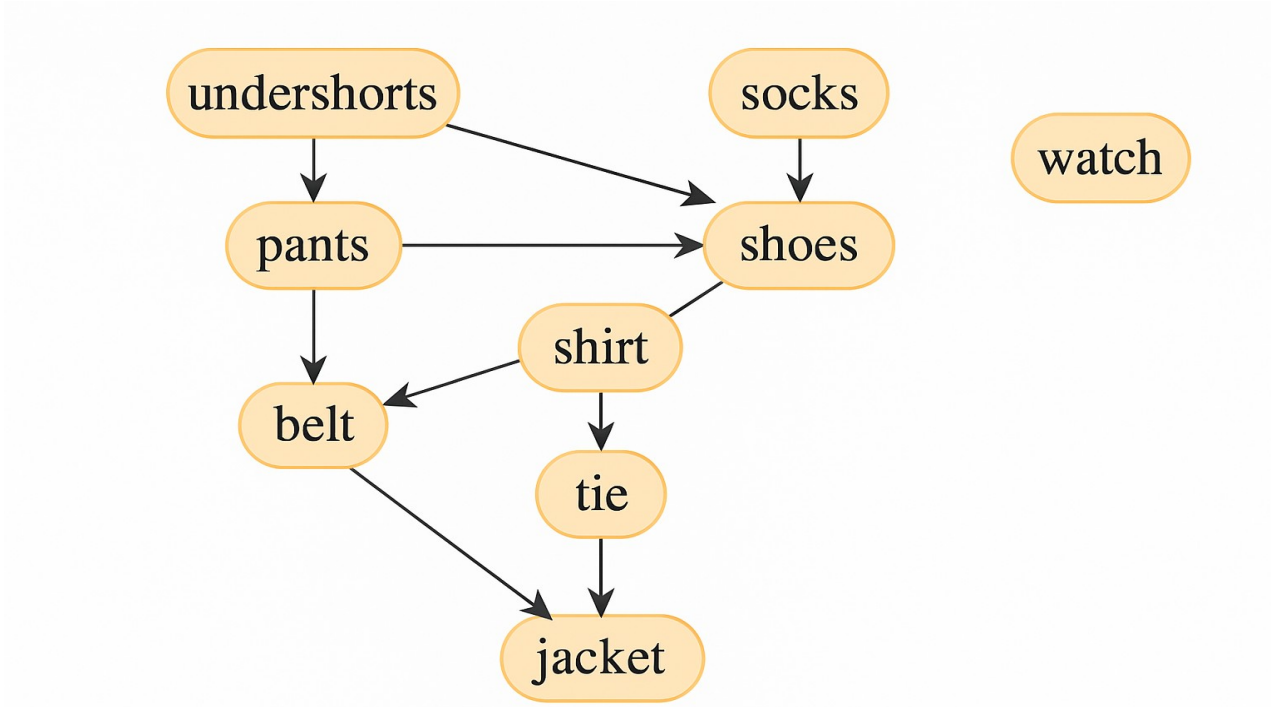
A directed graph with no cycles.

Good for modeling processes and structures that have a ***partial order:***

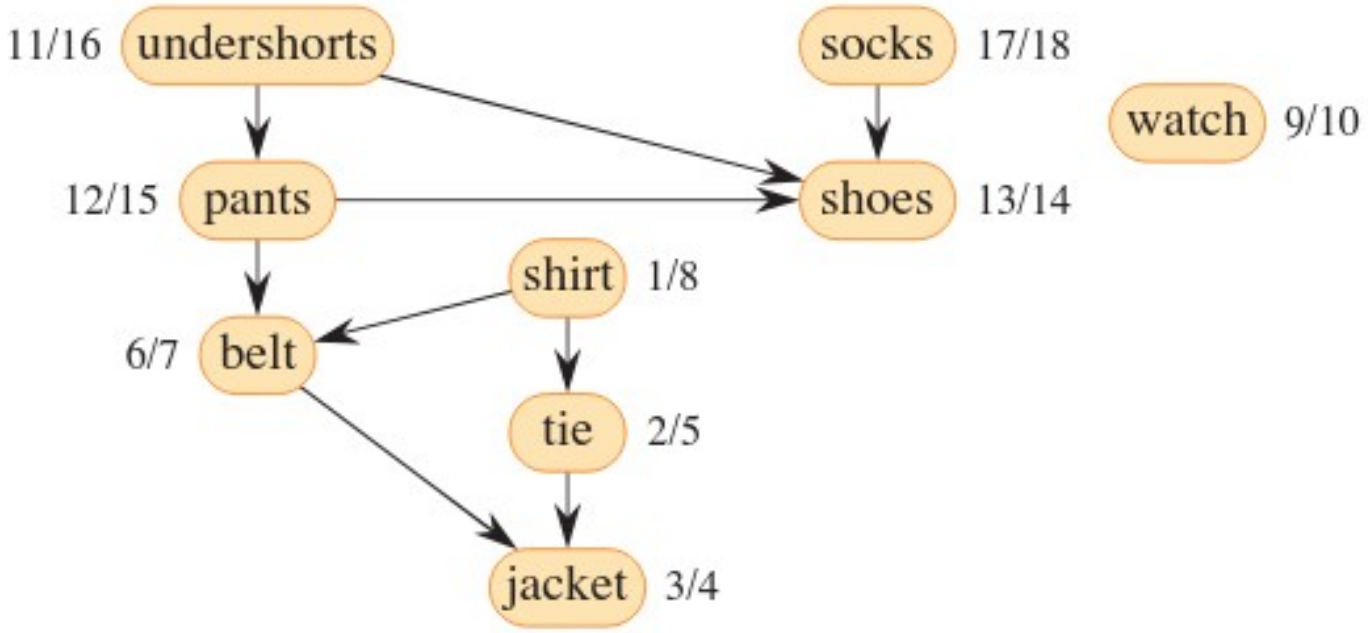
- $a > b$ and $b > c \Rightarrow a > c$.
- But may have a and b such that neither $a > b$ nor $b > c$.

Can always make a ***total order*** (either $a > b$ or $b > a$ for all $a \neq b$) from a partial order. In fact, that's what a topological sort will do.

EXAMPLE (dressing up)

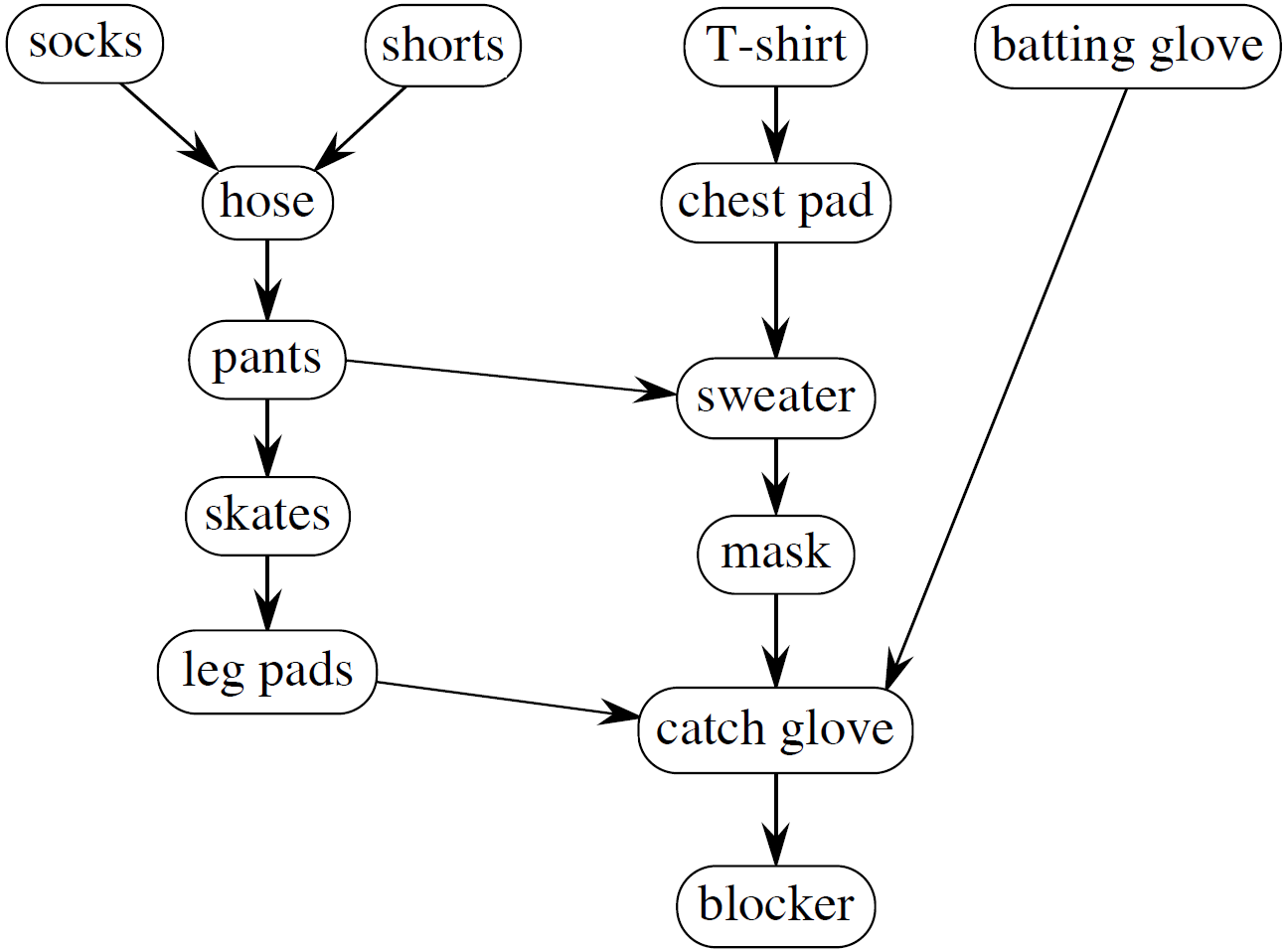


EXAMPLE (dressing up)



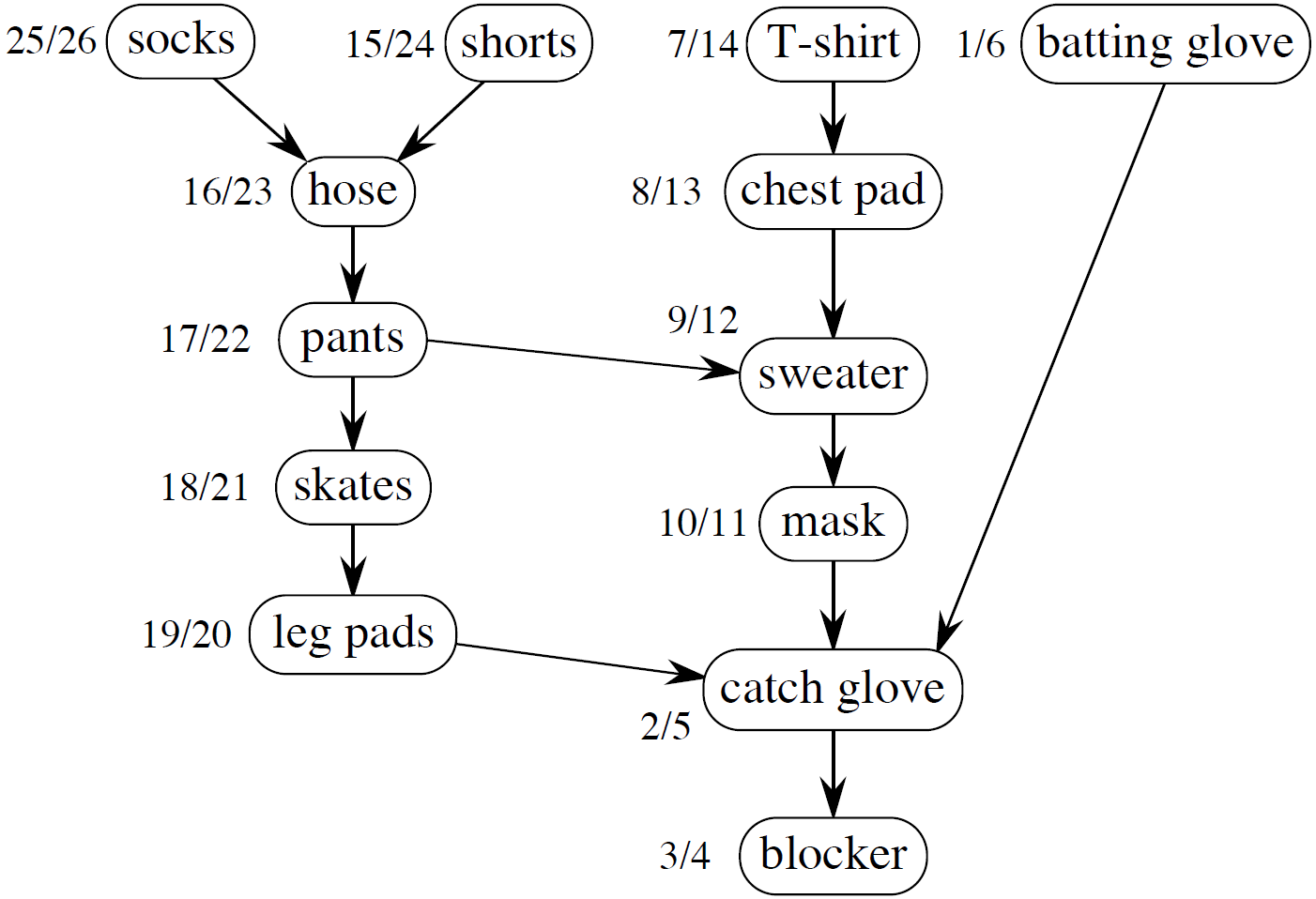
EXAMPLE

Dag of dependencies for putting on goalie equipment for ice hockey:



Discovery and Finish times

Dag of dependencies for putting on goalie equipment for ice hockey:



Ordering



Suppose there is an edge from A to X, is it a good ordering?

Ordering



Suppose there is an edge from A to X, is it a good ordering?

Can you make a good ordering?

Ordering



Suppose there is an edge from A to X, is it a good ordering?

Can you make a good ordering?

When can you make such good orderings?

Topological Ordering



Suppose there is an edge from A to X, is it a good ordering?

Can you make a good ordering?

When can you make such good orderings?

Topological sort of a dag: a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v. (Not like sorting numbers.)

DAG

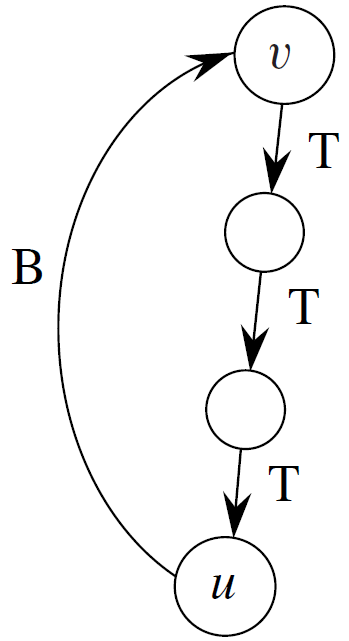
A directed graph G is acyclic if and only if a DFS of G yields no back edges.

DAG

A directed graph G is acyclic if and only if a DFS of G yields no back edges.

Proof $\Rightarrow$: Show that back edge $\Rightarrow$ cycle.

Suppose there is a back edge (u, v). Then v is ancestor of u in depth-first forest.



Therefore, there is a path $v \rightsquigarrow u$, so $v \rightsquigarrow u \rightarrow v$ is a cycle.

Reverse direction

A directed graph G is acyclic if and only if a DFS of G yields no back edges.

Proof:

No back edges in a DFS of G => G is acyclic.

equivalently,

G has a cycle => there is a back edge in a DFS of G

Reverse direction

A directed graph G is acyclic if and only if a DFS of G yields no back edges.

Proof:

G has a cycle => there is a back edge in a DFS of G

Suppose there is a cycle and let **u** be the vertex on the cycle that is discovered first.

White-path Theorem

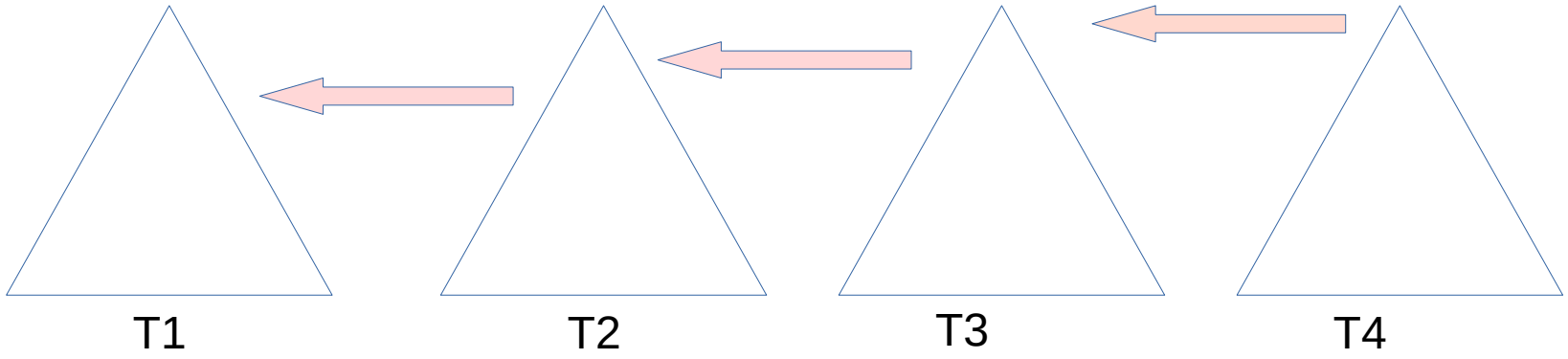
In a depth-first forest of a (directed or undirected) graph $G = (V, E)$, vertex v is a descendant of vertex u if and only if at the time $u.d$ that the search discovers u, there is a path from u to v consisting entirely of white vertices.

Topological Sort

Topological sort of a dag: a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v. (Not like sorting numbers.)

Topological Sort

Topological sort of a dag: a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v. (Not like sorting numbers.)



Ordering among the blocks?

Ordering in each block?

Topological Sort

Topological sort of a dag: a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v. (Not like sorting numbers.)

TOPOLOGICAL-SORT(G)

1 call DFS(G) to compute finish times $v.f$ for each vertex v
2 as each vertex is finished, insert it onto the front of a linked list
3 **return** the linked list of vertices

Topological Sort

Topological sort of a dag: a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v. (Not like sorting numbers.)

TOPOLOGICAL-SORT(G)

1 call DFS(G) to compute finish times $v.f$ for each vertex v
2 as each vertex is finished, insert it onto the front of a linked list
3 **return** the linked list of vertices

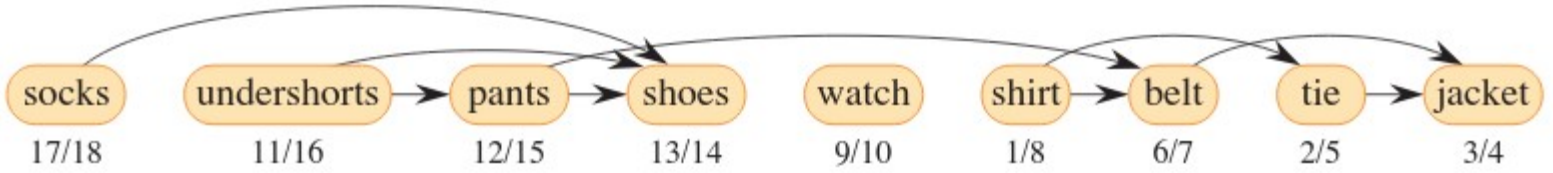
Don't need to sort by finish times.

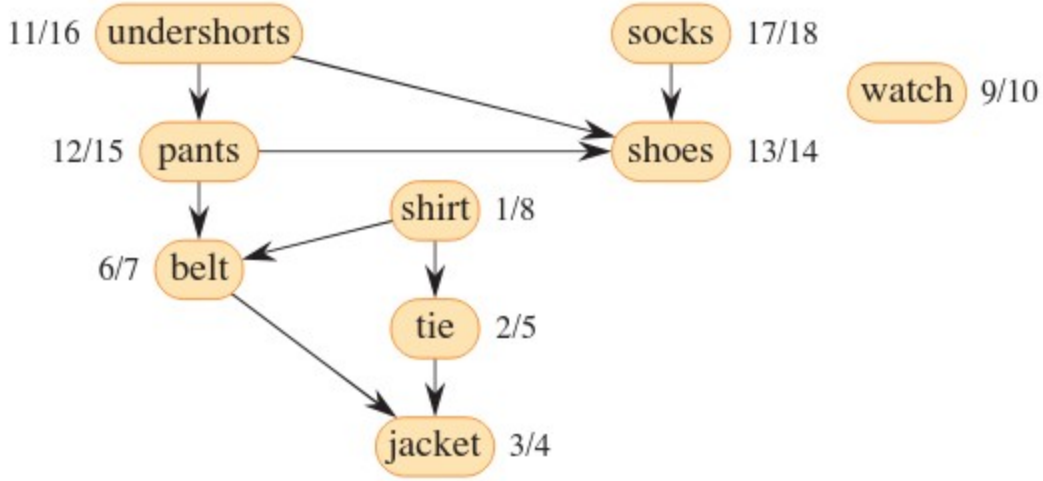
- Can just output vertices as they're finished and understand that we want the *reverse* of this list.
- Or put them onto the *front* of a linked list as they're finished. When done, the list contains vertices in topologically sorted order.

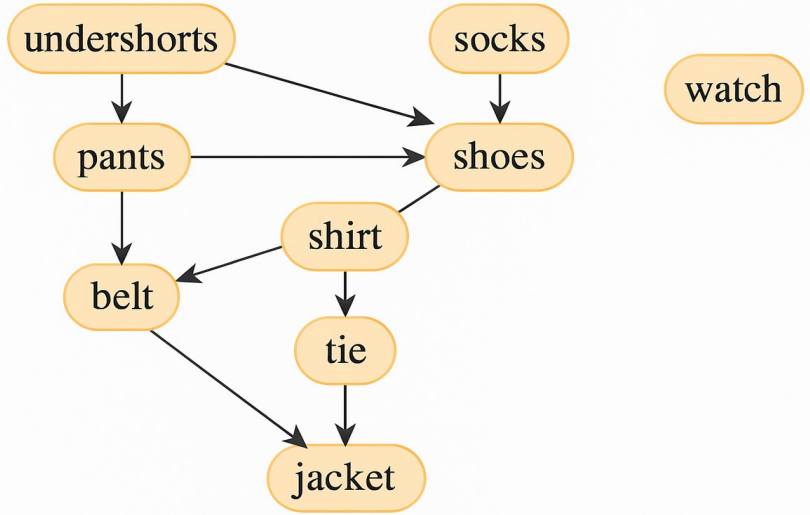
Time $\Theta(V + E)$.

Topological Sort

Let’s run DFS on this example to find finish times.

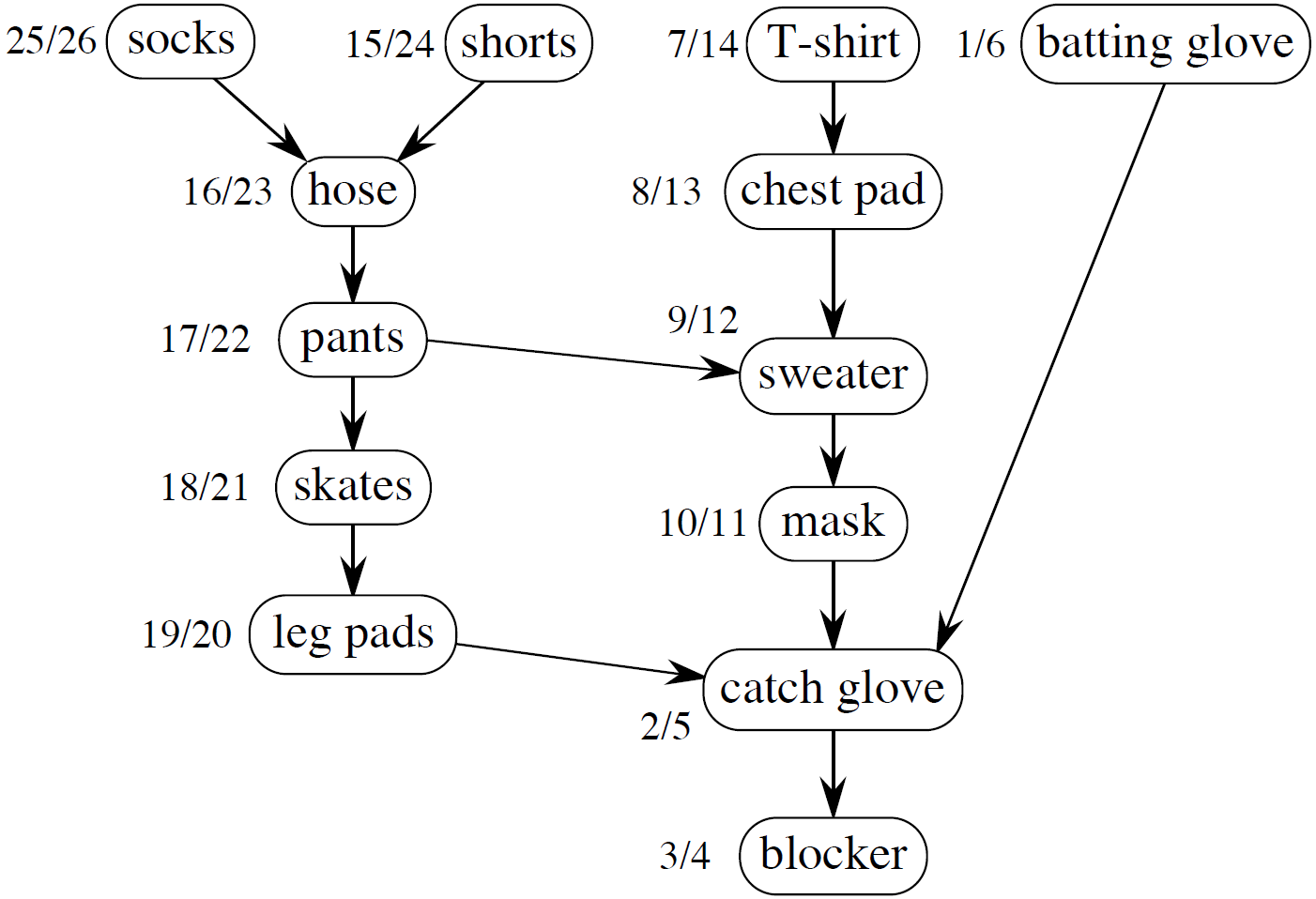






Topological Sort

Let's run DFS on this example to find finish times.



Correctness

Just need to show if $(u, v) \in E$, then $v.f < u.f$.
When edge (u, v) is explored, what are the colors of u and v?

Correctness

Just need to show if $(u, v) \in E$, then $v.f < u.f$.
When edge (u, v) is explored, what are the colors of u and v?

- u is gray.
- Is v gray, too?

Correctness

Just need to show if $(u, v) \in E$, then $v.f < u.f$.
When edge (u, v) is explored, what are the colors of u and v?

- u is gray.
- Is v gray, too?
  - *No*, because then v would be ancestor of u.
    $\Rightarrow (u, v)$ is a back edge.
    $\Rightarrow$ contradiction of previous lemma (dag has no back edges).
- Is v white?

Correctness

Just need to show if $(u, v) \in E$, then $v.f < u.f$.
When edge (u, v) is explored, what are the colors of u and v?

- u is gray.
- Is v gray, too?
  - *No*, because then v would be ancestor of u.
    $\Rightarrow (u, v)$ is a back edge.
    $\Rightarrow$ contradiction of previous lemma (dag has no back edges).
- Is v white?
  - Then becomes descendant of u.
    By parenthesis theorem, $u.d < v.d < v.f < u.f$.
- Is v black?

Correctness

Just need to show if $(u, v) \in E$, then $v.f < u.f$.
When edge (u, v) is explored, what are the colors of u and v?

- u is gray.
- Is v gray, too?
  - *No*, because then v would be ancestor of u.
    $\Rightarrow (u, v)$ is a back edge.
    $\Rightarrow$ contradiction of previous lemma (dag has no back edges).
- Is v white?
  - Then becomes descendant of u.
    By parenthesis theorem, $u.d < v.d < v.f < u.f$.
- Is v black?
  - Then v is already finished.
    Since exploring (u, v), u is not yet finished.
    Therefore, $v.f < u.f$.

Topological Sort

Topological sort of a dag: a linear ordering of vertices such that if $(u, v) \in E$, then u appears somewhere before v. (Not like sorting numbers.)

One can use BFS to obtain a topological sorting. It is known in literature as Kahn's Algorithm.